

Prüfung Web-Grundlagen

Lösungen

Frage 1

Der Statuscode **504 (Gateway Timeout)** ist ein Server-Fehler (5xx). Der Server (bzw. ein vorgeschalteter Loadbalancer oder Proxy) hat nicht rechtzeitig geantwortet und stattdessen eine HTML-Fehlerseite zurückgeliefert. `response.json()` versucht, diese HTML-Seite als JSON zu parsen: das erste Zeichen von HTML ist `<`, daher `SyntaxError: Unexpected token '<'`.

Verbesserung: Vor dem Parsen den Statuscode prüfen:

```
fetch('https://shop.example.com/api/products')
  .then((response) => {
    // vor dem Parsen response gültigkeit checken
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }
    return response.json();
  })
  .then((data) => { /* ... */ })
  .catch((error) => console.error('Fehler beim Laden:', error));
```

Frage 2

DNS (Domain Name System) ist das "Telefonbuch des Internets". Es übersetzt menschenlesbare Domain-Namen (z.B. `www.shop.com`) in IP-Adressen (z.B. `93.184.216.34`) und umgekehrt.

Rolle im Web: Ohne DNS müsste man sich IP-Adressen merken, um Websites zu erreichen. DNS-Server sind hierarchisch organisiert und lösen einen Domain-Namen schrittweise auf, bis die zugehörige IP-Adresse gefunden wird. Erst nach der Namensauflösung kann der Browser eine HTTP-Verbindung zum Server aufbauen.

Frage 3

Eigenschaft	Block-Level	Inline
Zeilenverhalten	Beginnt immer in neuer Zeile	Bleibt im Textfluss
Breite	Volle Breite des Containers	Nur so breit wie Inhalt
Beispiele	<code><div>, <p>, <h1>, <section>, </code>	<code>, <a>, , , </code>

Verschachtelungsregel: Inline-Elemente dürfen **keine** Block-Level-Elemente enthalten. Umgekehrt dürfen Block-Level-Elemente Inline-Elemente enthalten. `<p>` ist eine Ausnahme: es darf keine anderen Block-Level-Elemente enthalten.

Frage 4

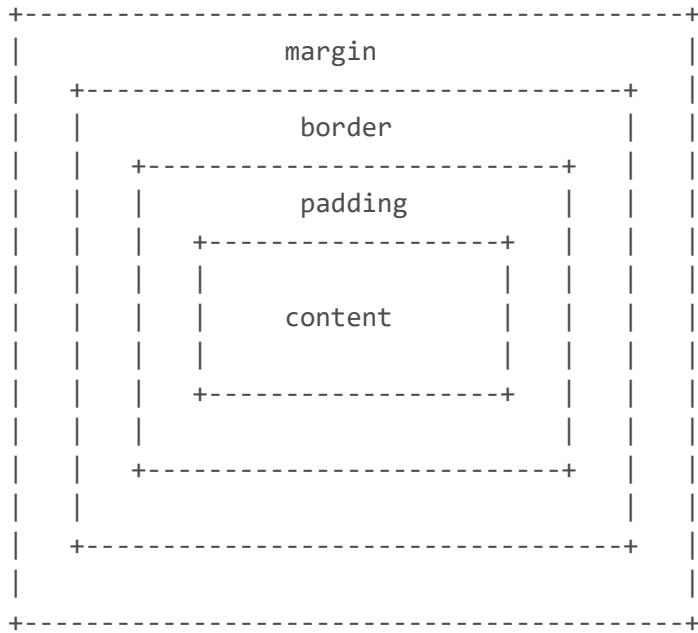
`https://www.shop.com/pages/kontakt.html?lang=de#top`

Komponente	Wert	Beschreibung
Schema (Protokoll)	<code>https</code>	Gibt das Übertragungsprotokoll an (hier: HTTPS, verschlüsselt)
Host (Domain)	<code>www.shop.com</code>	Domain-Name des Servers
Path (Pfad)	<code>/pages/kontakt.html</code>	Verzeichnis und Datei auf dem Server
Query	<code>?lang=de</code>	Query-Parameter als Key-Value-Paar (<code>lang=de</code>)
Fragment (Anchor)	<code>#top</code>	Verweis auf einen Teil innerhalb der Ressource

Hinweis: Bei HTTPS ohne explizite Port-Angabe wird implizit Port **443** verwendet.

Frage 5

Das **CSS Box-Modell** beschreibt, wie jedes HTML-Element als rechteckige Box dargestellt wird. Jede Box besteht aus vier Schichten:



- **border:** Rahmen um das Element, zwischen padding und margin
- **content:** Der eigentliche Inhalt (Text, Bild etc.)
- **margin:** Außenabstand zum nächsten Element
- **padding:** Innenabstand zwischen content und border

Mit **box-sizing: border-box** werden padding und border in die angegebene Breite/Höhe eingerechnet (Standardverhalten mit **content-box**: Breite bezieht sich nur auf content).

Frage 6

- **Paragraph 1: grün** — `#wrapper p` (0,1,0,1) schlägt `.container p` (0,0,1,1)
 - **Paragraph 2: cyan** — `#wrapper .container p.urgent` (0,1,1,2) schlägt alle anderen Regeln
 - **Paragraph 3: magenta** — Inline-Style hat höchste Priorität
 - **Paragraph 4: gelb** — Inline-Style überschreibt alle Stylesheet-Regeln
-

Frage 7

Zwei Vorteile von `<div>` gegenüber semantischen Elementen:

1. **Semantische Neutralität:** Ein `<div>` hat keine eingebaute semantische Bedeutung. Es beeinflusst weder Screenreader noch SEO. Wenn ein Element nur als Styling-Container oder Layout-Hilfe dient (z.B. reiner Wrapper für Flexbox/Grid), ist `<div>` die korrekte Wahl. Ein semantisches Element würde hier falsche Bedeutung vortäuschen.
2. **Flexibilität ohne ARIA-Konflikte:** Semantische Elemente haben implizite ARIA-Rollen. Ein `<section>` z.B. erwartet eine Überschrift, sonst gibt der Validator Warnungen aus. `<div>` hat keine solche Erwartung und eignet sich daher für generische Container ohne spezifische Dokumentstruktur-Rolle.

Wann `<div>` die bessere Wahl ist: Reine Layout-Container, Styling-Wrapper, oder wenn kein passendes semantisches Element existiert.

Frage 8

- **Fehlertyp: Client-Fehler** (4xx)
- **Statuscode 401 (Unauthorized):** Die Anfrage erfordert eine gültige Authentifizierung, die fehlt oder ungültig ist.
- **Wahrscheinliche Ursache:** Der API-Aufruf enthält keinen gültigen Authentifizierungs-Token (z.B. nicht mitgesendet, abgelaufen oder falsch formatiert). Häufig fehlt der `Authorization`-Header.
- **Behebung:** Sicherstellen, dass vor dem API-Aufruf ein gültiger Token vorhanden ist (z.B. aus Login, LocalStorage) und im Request-Header mitgesendet wird:

```
fetch('https://shop.example.com/api/orders', {  
  method: 'POST',  
  headers: {  
    'Authorization': `Bearer ${token}`,  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify(orderData)  
});
```

Falls der Token abgelaufen ist: Benutzer zur erneuten Anmeldung weiterleiten (Refresh-Token-Mechanismus).

Frage 9

- **Fehlertyp: Server-Fehler** (5xx)
- **Statuscode 500 (Internal Server Error):** Ein unerwarteter Fehler ist serverseitig aufgetreten.
- **Liegt am eigenen Code?** Nein, die URL ist korrekt, der Request wurde erfolgreich an den Server gesendet (im Sinne der Netzwerk-Kommunikation). Der Fehler entsteht bei der Verarbeitung auf dem Server (z.B. Datenbank nicht erreichbar, Programmierfehler im Backend, fehlgeschlagene Validierung).
- **Nächste Schritte:**
 1. Prüfen, ob der API-Endpunkt generell erreichbar ist
 2. Server-Logs des Backends prüfen (als Frontend-Entwickler: Backend-Team informieren)
 3. Im eigenen Code: Benutzerfreundliche Fehlermeldung anzeigen statt leerer/unerwarteter Zustand

```
.catch((error) => {  
  console.error('Fehler beim Laden der Produkte:', error);  
  document.getElementById('error').textContent =  
    'Produkte konnten nicht geladen werden. Bitte versuchen Sie es später erneut.';  
});
```

Frage 10

Fehler: Der `<script>`-Tag ist nicht geschlossen. Es fehlt `</script>`.

Warum leere Seite: Der Browser interpretiert alles nach dem öffnenden `<script>` als JavaScript-Code, bis zum ersten `</script>`, das nie kommt. Dadurch werden `<head>`-Ende, `<body>`, `<h1>`, `<p>`, `</body>` und `</html>` alle als Script-Inhalt aufgefasst. Das Dokument hat für den Renderer keinen sichtbaren Body-Inhalt.

Behebung: Schließenden `</script>`-Tag einfügen:

```
<head>  
  <meta charset="UTF-8">  
  <title>Meine Seite</title>  
  <script>  
    console.log("Initialisierung...");  
  </script>  
</head>
```